

Matriz

Sebas y Saúl son criaturas bidimensionales simples que estaban pensando en el siguiente problema:

Sebas le quiere transmitir un mensaje a Saúl, que es un arreglo a de tamaño n ($1 \leq n \leq 100$ y $1 \leq a_i \leq 10^4$), a través de una matriz binaria (cuyos valores son 0 o 1) de 100×100 . Ambos se pusieron de acuerdo para resolver el problema, tal que sin importar que matriz mande Sebas, Saúl pueda reconstruir correctamente el arreglo original de Sebas sin alterar su orden.

Implementaron una solución en Karel, intentando minimizar la cantidad de posiciones de la matriz que manda Sebas que eran iguales a 1. Tu reto es resolver este problema de una forma más eficiente.

Detalles de Implementación

Deberás implementar dos funciones en el mismo archivo.

La función `sebas()` deberá implementar tu estrategia para Sebas.

```
vector<vector<int>> sebas(int n, vector<int> a);
```

Esta función recibe dos parámetros:

- Un entero n , el tamaño del arreglo ($1 \leq n \leq 100$).
- El arreglo a a codificar, un vector de enteros a (con $1 \leq a_i \leq 10^4$).

Y regresa un vector de vectores de enteros, la matriz binaria que manda Sebas. En particular, tu código debe regresar un vector de 100 vectores de tamaño 100, cuyos elementos sean todos 1 o 0. Si esto no se cumple, recibirás un veredicto de respuesta incorrecta.

La función `saul()` deberá implementar tu estrategia para Saúl.

```
vector<int> saul(vector<vector<int>> matriz);
```

Esta función recibe un parámetro:

- Un vector de vectores de enteros, `matriz` (se garantiza que tiene dimensiones 100×100 y todos los elementos son 1 o 0).

Y regresa un vector de enteros, que debe corresponder al vector que codificó Sebas.

A continuación se muestra un ejemplo de cómo se vería el código que envías (**revisa los archivos adjuntos, ahí también encontrarás un archivo que emula el evaluador para poder probar tu código**).

```
#include <bits/stdc++.h>
using namespace std;

vector<vector<int>> sebas(int n, vector<int> a) {
    // programa tu solución aquí.
}

vector<int> saul(vector<vector<int>> matriz) {
    // programa tu solución aquí.
}
```

El evaluador correrá tu programa dos veces por caso. En la primera, llamará la función *sebas()* múltiples veces, una para cada subcaso de prueba (**recuerda reinicializar las variables globales**). En la segunda, llamará la función *saul()* múltiples veces, una por cada subcaso de prueba (**recuerda reinicializar las variables globales**).

Ejemplo

- El evaluador corre tu programa, y llama la función: *sebas*(4, {3, 10, 5, 2})
- El evaluador guarda la matriz que regresa, llamémosla *M*, y finaliza la ejecución.
- Luego, el evaluador vuelve a correr tu programa, y llama la función *saul()* con *M* como parámetro.
- Si el arreglo que recibe el evaluador es exactamente {3, 10, 5, 2}, entonces le asigna los puntos correspondientes dependiendo de cuantas entradas con 1s tenga la matriz *M*. De lo contrario, da veredicto de respuesta incorrecta.

Consideraciones

- Tu programa será ejecutado por el evaluador **dos veces por caso**, por lo que todas las variables globales que guardes en la primera ejecución se perderán en la segunda.
- La función *sebas()* debe responder un vector de vectores de enteros, de tamaño 100×100 . Cada valor debe ser 0 o 1. De lo contrario recibirás un veredicto de respuesta incorrecta.
- La función *saul()* debe responder con un vector de enteros que produce la matriz al llamarse *sebas()* con el vector como argumento. Cada valor *i* del vector debe cumplir $1 \leq i \leq 10^4$. De lo contrario recibirás un veredicto de respuesta incorrecta.
- Para todos los casos, se cumple que $1 \leq n \leq 100$.

Subtareas

- Subtarea 1 (5 puntos): Para todo i , $1 \leq a_i \leq 100$ y no hay límite sobre la cantidad de posiciones en la matriz que pueden ser 1.

Todas las subtareas a excepción de la primera, otorgan puntos parciales si la cantidad de posiciones iguales a 1 en la matriz está en algún rango. Para cada subtarea, digamos que X es la máxima cantidad de posiciones de la matriz que son iguales a 1 para algún caso.

- Subtarea 2 (12.5-25 puntos): La cantidad de posiciones iguales a 1 en la matriz está entre $7N$ y $14N$ (inclusive).
 - Si $14N < X$, recibirás 0 puntos.
 - Si X es menor o igual $7N$ recibirás 25 puntos.
 - si $7N < X < 14N$, recibirás

$$12.5 + 12.5 \left(\frac{14N - X}{14N - 7N} \right) \text{ puntos.}$$

- Subtarea 3 (0-25 puntos): La cantidad de posiciones iguales a 1 en la matriz está entre $3N$ y $7N$ (inclusive).
 - Si $7N < X$, recibirás 0 puntos.
 - Si X es menor o igual $3N$ recibirás 25 puntos.
 - si $3N < X < 7N$, recibirás

$$25 \left(\frac{7N - X}{7N - 3N} \right) \text{ puntos.}$$

- Subtarea 4 (0-30 puntos): La cantidad de posiciones iguales a 1 en la matriz está entre $2N$ y $3N$ (inclusive). Además, se garantiza que los números en el arreglo a codificar están ordenados de menor a mayor.
 - Si $3N < X$, recibirás 0 puntos.
 - Si X es menor o igual $2N$ recibirás 30 puntos.
 - si $2N < X < 3N$, recibirás

$$30 \left(\frac{3N - X}{3N - 2N} \right) \text{ puntos.}$$

- Subtarea 5 (0-15 puntos): La cantidad de posiciones iguales a 1 en la matriz está entre $2N$ y $3N$ (inclusive).
 - Si $3N < X$, recibirás 0 puntos.

- Si X es menor o igual $2N$ recibirás 15 puntos.
- si $2N < X < 3N$, recibirás

$$15 \left(\frac{3N - X}{3N - 2N} \right) \text{ puntos.}$$

Evaluador de prueba

En el apartado de *Adjuntos* vendrá un documento llamado *grader.cpp*. dicho documento ayudará a simular nuestro código.

Enunciado

no hay ningún enunciado disponible

Algunos detalles

Tipo	Communication
Límite de tiempo	3,000 segundos
Límite de memoria	1,00 GiB
Comandos de compilación	C++20 / g++ /usr/bin/g++ -DEVAL -std=gnu++20 -O2 -pipe -static -s -o matriz_stub.cpp matriz.cpp C11 / gcc /usr/bin/gcc -DEVAL -std=gnu11 -O2 -pipe -static -s -o matriz_stub.c matriz.c -lm Pascal / fpc /usr/bin/fpc -dEVAL -O2 -Xs -omatrix_stub.pas

Adjuntos

 **grader.cpp** 961 bytes
C++ source code

Dentro de dicho documento, nos encontraremos con código que contiene la función que deberemos programar.

```

Start here X matriz.cpp X grader.cpp X
1 #include <bits/stdc++.h>
2 using namespace std;
3
4
5
6
7
8
9 // TU CODIGO EMPIEZA AQUI
10
11 vector<vector<int>>> sebas(int n, vector<int> a) {
12     vector<vector<int>>> matriz(100, vector<int>(100, 0)); // Esta matriz ya es de tamaño 100 x 100 y está llena de ceros
13
14     return matriz;
15 }
16
17 vector<int> saul(vector<vector<int>>> matriz) {
18     vector<int> ans;
19
20     return ans;
21 }
22
23 // TU CODIGO TERMINA AQUI
24
25
26
27
28
29

```

No hace falta preocuparse por todo lo que esté fuera del rectángulo rojo, abajo son las funciones que usa el programa para calificar nuestro código.

Cuando queramos probar nuestro código solo deberemos correr el documento completo. El documento leerá un entero N , seguido de N enteros $a_1, a_2, a_3 \dots, a_n$, los cuales son los elementos del arreglo que recibe Sebas.

```
// TU CODIGO EMPIEZA AQUI
vector<vector<int>> sebas(int n, vector<vector<int>> matriz) {
    return matriz;
}

vector<int> saul(vector<vector<int>> matriz) {
    vector<int> ans;
    ans.push_back(1);
    return ans;
}
// TU CODIGO TERMINA AQUI
```

```
5
1 2 4 2 1
WA: El arreglo retornado no es del mismo tamaño que el que recibe Sebas :(
Process returned 0 (0x0)   execution time : 4.472 s
Press any key to continue.
```

Si le damos una entrada válida, el documento nos dirá si nuestro código generó una respuesta correcta ó errónea, además del número de bits que le tomó a nuestro código obtener dicha respuesta.

Una vez que tengamos nuestro código finalizado, deberemos copiar lo que escribimos dentro de los “TU CODIGO VA AQUI” y pegarlo en un nuevo documento.